

Agile Methods – Unidade II



Oi, pessoal!

No mundo da tecnologia, os princípios do **Lean** e as práticas de **Extreme Programming (XP)** são fundamentais para otimizar o desenvolvimento de software e aumentar a produtividade das equipes. Enquanto o Lean se foca na eliminação de desperdícios e na melhoria contínua dos processos, o XP se concentra em práticas ágeis que promovem a entrega de software de alta qualidade. Essas metodologias são amplamente usadas em empresas de software e oferecem aos profissionais a base para desenvolver soluções de forma mais eficiente e com menor taxa de erros.

Ao dominar os princípios do **Lean** e as práticas de **Extreme Programming (XP)**, os estudantes estarão preparados para enfrentar os desafios do desenvolvimento de software moderno. Eles aprenderão a otimizar processos, reduzir desperdícios e melhorar a qualidade do código por meio de técnicas como programação em pares, testes unitários e integração contínua. O conhecimento dessas metodologias será um diferencial competitivo no mercado de trabalho e um aprendizado enriquecedor.

Apertem os cintos e vamos decolar! 🚀



Aula 1 - Introdução ao Lean: Origens no Sistema Toyota de Produção



Aula 2 - O que São testes Unitários e por que São Importantes



Download de Conteúdo

Aula 1 – Introdução ao Lean: Origens no Sistema Toyota de Produção

Videoaula 5 – Princípios do Lean: Como eliminar desperdícios no desenvolvimento de software

Explore os princípios do Lean e descubra como aplicá-los para reduzir desperdícios e otimizar processos no desenvolvimento de software. Esta aula oferece insights sobre como tornar as operações mais enxutas e orientadas para a entrega de valor.

Pronto para essa aventura? Aperte o play e bons estudos! 🎮🚀

07:53

QUAL É A IMPORTÂNCIA PARA SUA ATUAÇÃO PROFISSIONAL?

Compreender os princípios do Lean permite aos estudantes desenvolver uma mentalidade de otimização contínua dos processos de trabalho. Identificar e eliminar desperdícios, como retrabalho ou espera entre fases de desenvolvimento, melhora a eficiência e a qualidade final dos produtos. O conceito de Kaizen ensina como pequenas mudanças incrementais podem gerar grandes resultados ao longo do tempo. Ao aplicar esses princípios no desenvolvimento de software, os profissionais estarão mais aptos a entregar soluções mais rapidamente e com menos erros.

Introdução ao Lean: Origens no Sistema Toyota de Produção

O Lean é uma filosofia de gestão derivada do sistema de produção da Toyota, que tem como objetivo **eliminar desperdícios** e melhorar continuamente os processos. Essa abordagem foi inicialmente desenvolvida para otimizar a produção de automóveis, mas suas ideias foram rapidamente aplicadas em outros setores, incluindo o desenvolvimento de software. No contexto do desenvolvimento ágil, o Lean se concentra em maximizar o valor entregue ao cliente enquanto minimiza o tempo e os recursos desperdiçados ao longo do processo.

Os 7 tipos de desperdício no Lean e sua aplicação no desenvolvimento de software

No Lean, existem sete tipos de desperdício que devem ser eliminados para garantir a máxima eficiência. **Esses desperdícios são:**

1. Superprodução

Produzir mais do que o necessário ou antes do tempo necessário. No desenvolvimento de software, isso pode significar escrever código ou criar funcionalidades que não são imediatamente necessárias ou que nunca serão usadas.

2. Espera

Tempo ocioso quando os processos estão parados. Em software, isso pode ocorrer quando os desenvolvedores estão esperando por feedback, aprovação ou recursos.

3. Transporte

Movimentação desnecessária de materiais ou informações. No contexto de software, isso pode se referir à transferência de dados entre sistemas ou equipes que não agrega valor ao produto final.

4. Excesso de Processamento

Fazer mais trabalho ou usar mais recursos do que o necessário. Isso pode incluir a criação de documentação excessiva ou a implementação de funcionalidades complexas que não são essenciais.

5. Inventário

Acúmulo de trabalho em progresso ou produtos não utilizados. Em software, isso pode ser visto como código não integrado ou funcionalidades parcialmente desenvolvidas que não estão em uso.

6. Movimentação

Movimentação desnecessária de pessoas. Para desenvolvedores, isso pode significar a necessidade de mudar frequentemente de contexto ou de ferramentas, o que reduz a produtividade.

7. Defeitos

Erros ou falhas que exigem retrabalho. No desenvolvimento de software, isso se traduz em bugs e problemas de qualidade que precisam ser corrigidos, consumindo tempo e recursos adicionais.

Ao identificar e eliminar esses desperdícios, as equipes de desenvolvimento de software podem aumentar significativamente a eficiência, reduzir custos e melhorar a qualidade do produto final. A aplicação dos princípios Lean no desenvolvimento ágil ajuda a

garantir que o foco permaneça na entrega de valor ao cliente de forma contínua e eficiente.

Benefícios da Mentalidade Lean nas Equipes Ágeis

A adoção da mentalidade Lean nas equipes ágeis traz diversos benefícios, incluindo:



Maior Eficiência: Redução de desperdícios e otimização dos processos de desenvolvimento.



Melhoria da Qualidade: Foco na entrega de valor e na satisfação do cliente, resultando em produtos de maior qualidade.



Flexibilidade e Adaptabilidade: Capacidade de responder rapidamente às mudanças e ajustar o curso conforme necessário.



Engajamento da Equipe: Equipes mais motivadas e engajadas, com maior autonomia e responsabilidade.

Kaizen: Melhoria contínua e eliminação de desperdícios

O conceito de **Kaizen**, que significa "mudança para melhor" em japonês, é um pilar do Lean. Ele promove a **melhoria contínua** em pequenos incrementos, em vez de grandes mudanças radicais. No desenvolvimento ágil, o Kaizen pode ser implementado através de retrospectivas e ciclos de feedback constantes, onde a equipe identifica pontos de melhoria e ajusta seus processos para otimizar o fluxo de trabalho e eliminar desperdícios.

A mentalidade Lean nas equipes ágeis: adaptação ao ambiente de desenvolvimento

Adotar uma **mentalidade Lean** dentro de equipes ágeis significa sempre buscar formas de melhorar, eliminar desperdícios e aumentar a eficiência. Isso pode ser alcançado incentivando a

colaboração constante entre os membros da equipe, otimizando a comunicação e garantindo que todos estejam focados em entregar valor contínuo ao cliente.

A mentalidade Lean, originada no **Sistema Toyota de Produção**, tem sido amplamente adotada por equipes ágeis no desenvolvimento de software devido à sua ênfase na eliminação de desperdícios e na melhoria contínua. Essa abordagem se adapta perfeitamente ao ambiente de desenvolvimento ágil, onde a flexibilidade e a eficiência são cruciais.

Princípios Lean Aplicados ao Desenvolvimento Ágil

1. Eliminação de Desperdícios

No contexto ágil, desperdícios podem incluir código não utilizado, funcionalidades desnecessárias, e tempo de espera entre as etapas do processo. As equipes ágeis utilizam práticas como o desenvolvimento incremental e a integração contínua para minimizar esses desperdícios.

2. Melhoria Contínua (Kaizen)

As equipes ágeis realizam retrospectivas regulares para avaliar o que funcionou bem e o que pode ser melhorado. Esse ciclo de feedback constante permite ajustes rápidos e incrementais, alinhando-se com o princípio Lean de melhoria contínua.

3. Foco no Cliente

Tanto o Lean quanto o ágil colocam o cliente no centro do processo. As equipes ágeis trabalham em estreita colaboração com os clientes para garantir que o produto final atenda às suas necessidades e expectativas, entregando valor continuamente.

4. Empoderamento da Equipe

A mentalidade Lean promove a autonomia e a responsabilidade das equipes. No desenvolvimento ágil, isso se traduz em equipes auto-

organizadas que têm a liberdade de tomar decisões e ajustar seu trabalho conforme necessário para alcançar os melhores resultados.

5. Fluxo de Trabalho Eficiente

O Lean enfatiza a criação de um fluxo de trabalho suave e eficiente. As equipes ágeis utilizam ferramentas como Kanban e Scrum para visualizar e gerenciar o fluxo de trabalho, identificando e eliminando gargalos que possam atrasar a entrega de valor.

Videoaula 6 – Práticas de Extreme Programming (XP)

Conheça as práticas centrais do Extreme Programming (XP), um dos métodos ágeis focado em qualidade e flexibilidade no código. Nesta videoaula, abordaremos conceitos como refatoração, programação em pares e desenvolvimento orientado a testes, com ênfase na melhoria contínua.

Dê o play agora na videoaula a seguir! 🧠✨

11:24

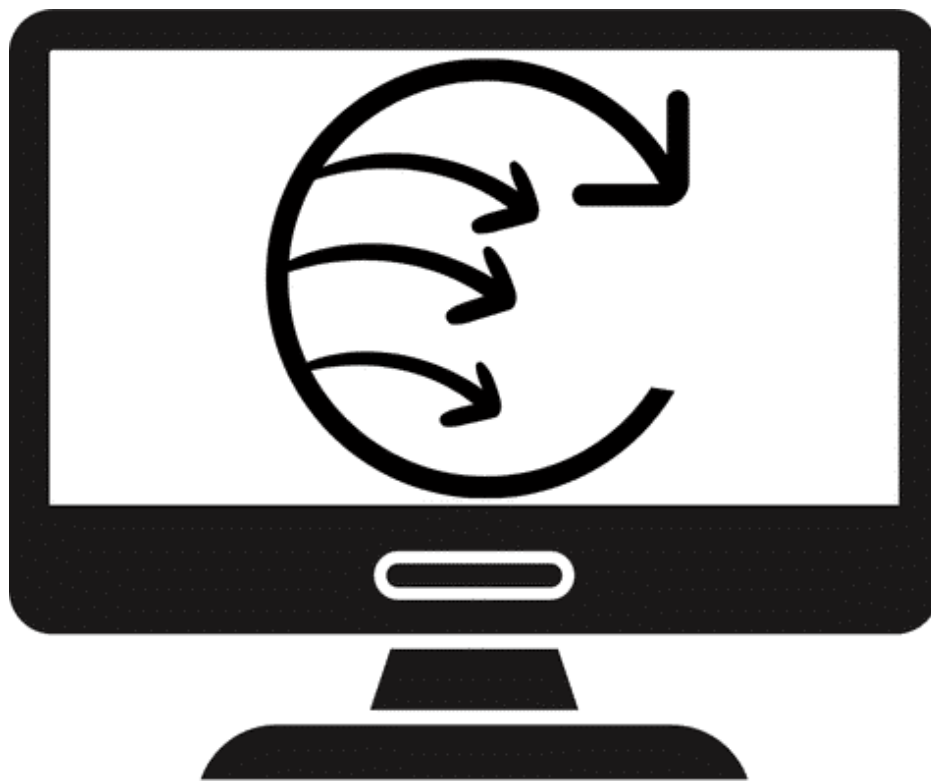
QUAL É A IMPORTÂNCIA PARA SUA ATUAÇÃO PROFISSIONAL?

As práticas de Extreme Programming (XP) são essenciais para quem deseja desenvolver software de alta qualidade. A programação em pares é uma das práticas mais conhecidas, onde dois desenvolvedores trabalham juntos em um código, promovendo troca de conhecimento e a detecção de erros em tempo real. Essa prática aumenta a qualidade do código e reduz o número de bugs. Além disso, os estudantes aprenderão sobre outras práticas de XP, como refatoração e

simplicidade, que ajudam a manter o código limpo e adaptável ao longo do tempo.

O Extreme Programming (XP) e como ele se diferencia de outras metodologias ágeis

O **Extreme Programming (XP)** é uma metodologia ágil que tem como foco a entrega contínua de software de alta qualidade por meio de práticas técnicas rigorosas. A XP se diferencia de outras metodologias ágeis, como Scrum, ao se concentrar em práticas específicas de desenvolvimento, como programação em pares, testes automatizados e refatoração contínua. Seu principal objetivo é garantir que o código entregue tenha qualidade e flexibilidade para mudanças.



1

Programação em pares: conceito e benefícios

A **programação em pares** é uma das práticas centrais do XP. Ela envolve dois desenvolvedores trabalhando juntos no mesmo código, em que um escreve enquanto o outro revisa. Esse processo promove uma troca constante de conhecimento, aumenta a qualidade do código ao identificar erros em tempo real e reduz a necessidade de retrabalho. A programação em pares também melhora a coesão da equipe, promovendo a colaboração.

2

Revisão de código em tempo real: como funciona na prática

A revisão de código em tempo real, facilitada pela programação em pares, permite que erros sejam corrigidos imediatamente, em vez de serem descobertos posteriormente por revisores ou testadores. Esse processo melhora significativamente a qualidade do software, pois as falhas são detectadas durante o desenvolvimento, o que reduz o custo e o tempo para corrigi-las.

3

Outras práticas de XP: refatoração, simplicidade e design evolutivo

Além da programação em pares, o XP incentiva outras práticas, como:

- **Refatoração** – a melhoria contínua do código sem alterar seu comportamento externo.
- **Simplicidade** – desenvolver o código mais simples possível que atenda aos requisitos do cliente.

- **Design evolutivo** – permitir que o design do sistema evolua ao longo do tempo à medida que os requisitos se tornam mais claros.

**Essas práticas ajudam a garantir
que o software seja flexível e fácil
de manter no longo prazo.**

Saiba mais

A Programação Extrema (XP) é uma metodologia ágil que se destaca pela sua flexibilidade e adaptabilidade no desenvolvimento de software. Ela prioriza a colaboração constante entre a equipe e o cliente, entregas frequentes e a qualidade do código, com práticas como programação em pares, testes automatizados e refatoração contínua. XP garante que o software esteja sempre evoluindo, com foco na melhoria contínua e na capacidade de responder rapidamente a mudanças, tornando-a uma escolha ideal para projetos dinâmicos e desafiadores.

XP Programming (COMECE POR A

Código Fonte TV



Watch on

Próxima Aula 

Aula 2 – O que São testes Unitários e por que São Importantes

Videoaula 7 – Testes Unitários no Desenvolvimento Ágil

Nesta videoaula, você aprenderá a importância dos testes unitários no contexto ágil, garantindo que cada parte do código funcione de forma independente. Abordaremos as melhores práticas para criação de testes, que ajudam a aumentar a confiabilidade e a qualidade do software.

Pronto para essa aventura? Aperte o play e bons estudos! 🎮🚀

21:51

QUAL É A IMPORTÂNCIA PARA SUA ATUAÇÃO PROFISSIONAL?

Os estudantes aprenderão a criar testes unitários, que são fundamentais para garantir a qualidade do software. Testes unitários verificam se pequenas partes do código funcionam corretamente de maneira isolada, permitindo que os desenvolvedores identifiquem problemas antes que se tornem grandes falhas. Além de aprender a implementar esses testes, vocês desenvolveram habilidades em automação de testes, aumentando a velocidade de desenvolvimento e reduzindo o tempo gasto com correções de bugs.

O que são testes unitários e por que são importantes

Os **testes unitários** são uma prática de desenvolvimento onde pequenas partes (ou "unidades") de um software são testadas individualmente para garantir que funcionam conforme o esperado. Eles são fundamentais no XP e no desenvolvimento ágil, pois permitem que os desenvolvedores detectem problemas em uma fase inicial do processo, evitando que pequenos erros se transformem em grandes falhas ao longo do ciclo de desenvolvimento.

Exemplos práticos de criação de testes unitários em diferentes linguagens de programação

Os testes unitários podem ser criados em praticamente qualquer linguagem de programação. Por exemplo:

- Em Python, bibliotecas como **unittest** e **pytest** são amplamente usadas.

- Em JavaScript, **Jest** é uma das ferramentas mais populares. Esses testes são executados de forma automatizada e verificam se cada parte do código funciona corretamente, testando diversos cenários para garantir que as funções e métodos não gerem resultados inesperados.

Ferramentas para automação de testes unitários

A automação de testes unitários é uma prática importante que ajuda a garantir que os testes sejam executados regularmente sem a necessidade de intervenção manual. Ferramentas como:

- **JUnit** (para Java),
- **PyTest** (para Python), e
- **NUnit** (para .NET)

Permitem que os testes sejam integrados em pipelines de Integração Contínua (CI), garantindo que o código seja testado automaticamente após cada alteração.

A importância dos testes no processo de desenvolvimento contínuo

Os testes unitários são essenciais para garantir que o software seja de alta qualidade, especialmente em um ambiente de **integração contínua**. Eles garantem que novas mudanças no código não causem regressões ou introduzam bugs. Ao utilizar testes automatizados, as equipes conseguem verificar rapidamente se o software continua funcionando como esperado, mesmo com alterações frequentes no código.

Código da atividade da aula

FERRAMENTAS: GOOGLE COLAB E GITHUB

Código de teste:

```
!pip install pytest
```

```
with open("test_calculadora.py", "w") as f:  
    f.write("""  
import pytest
```

```
# Funções que serão testadas  
def soma(a, b):  
    return a + b
```

```
def subtracao(a, b):  
    return a - b
```

```
def multiplicacao(a, b):  
    return a * b
```

```
def divisao(a, b):  
    if b == 0:
```



```
    raise ValueError("Não pode dividir por zero!")  
    return a / b
```

```
# Testes
```

```
def test_soma():  
    assert soma(3, 2) == 6  
    assert soma(-1, 1) == 0
```

```
def test_subtracao():  
    assert subtracao(5, 3) == 2  
    assert subtracao(10, 10) == 0
```

```
def test_multiplicacao():  
    assert multiplicacao(3, 3) == 12  
    assert multiplicacao(4, -2) == -16
```

```
def test_divisao():  
  
    assert divisao(10, 2) == 5  
    with pytest.raises(ValueError):  
        divisao(10, 0)  
    """
```

```
!pytest test_calculadora.py
```

Videoaula 8 – Integração Contínua e Entrega Contínua

A integração contínua e a entrega contínua são práticas fundamentais para equipes ágeis que buscam otimizar o ciclo de desenvolvimento. Nesta videoaula, veremos como configurar e aplicar essas práticas, permitindo entregas frequentes e uma resposta mais rápida às mudanças e correções necessárias.

Dê o play agora na videoaula a seguir! 🎥✨

18:50

QUAL É A IMPORTÂNCIA PARA SUA ATUAÇÃO PROFISSIONAL? —

Vocês irão aprender a implementar pipelines de Integração Contínua (CI) e Entrega Contínua (CD), que são práticas essenciais para o

desenvolvimento ágil. A integração contínua permite que o código de todos os desenvolvedores seja mesclado várias vezes ao dia, garantindo que problemas de integração sejam detectados rapidamente. Já a entrega contínua automatiza o processo de disponibilização de novas versões do software para produção, garantindo que novos recursos cheguem aos usuários com agilidade.

Integração Contínua (CI) e como implementá-la em um pipeline de desenvolvimento

A **Integração Contínua (CI)** é uma prática de desenvolvimento que envolve a integração regular do código de todos os desenvolvedores em um repositório compartilhado. Cada integração é verificada por meio de testes automatizados, garantindo que o novo código funcione corretamente com o restante do sistema. Essa prática ajuda a identificar problemas de integração cedo, evitando grandes conflitos de código.

Ferramentas como **Jenkins**, **Travis CI** e **CircleCI** são amplamente utilizadas para automatizar a integração contínua.

Ferramentas para Integração Contínua

Essas ferramentas de CI automatizam o processo de integração e execução de testes.

- O **Jenkins** é uma ferramenta open-source amplamente usada que permite a configuração de pipelines personalizados para a integração contínua e automação de tarefas.
- **Travis CI** e **CircleCI** são plataformas baseadas em nuvem que oferecem uma configuração rápida e eficiente para projetos de código aberto e privado, facilitando a automação de testes e integração.

A importância da Entrega Contínua (CD) para entregar valor rápido aos clientes

A **Entrega Contínua (CD)** é uma extensão da Integração Contínua, onde as alterações de código são automaticamente preparadas para serem lançadas em produção. Isso garante que o software esteja sempre em um estado pronto para ser entregue ao cliente. A CD permite que as equipes entreguem novos recursos de forma rápida e confiável, reduzindo o tempo de lançamento e aumentando a capacidade de resposta às mudanças.

Como CI e CD reduzem riscos e aceleram o desenvolvimento ágil

A combinação de CI e CD permite que as equipes reduzam os riscos de integração e entrega de novas funcionalidades, aumentando a confiança no código e garantindo que ele possa ser liberado em produção com segurança. Além de aumentar a velocidade de entrega, essas práticas promovem a **agilidade** e a **resposta rápida** às demandas dos clientes, garantindo um ciclo de desenvolvimento contínuo e eficiente.

Código da atividade da aula

Código de teste:

Este é um fluxo de trabalho básico para ajudar você a começar com ações

nome : CI

Controla quando o fluxo de trabalho será executado sobre :

Aciona o fluxo de trabalho em eventos de solicitação push ou pull, mas apenas para o branch "principal"

empurrar :

ramos : ["principal"]

solicitação de pull :

ramos : ["principal"]

Permite que você execute este fluxo de trabalho manualmente na guia Ações

fluxo de trabalho_despacho :

Uma execução de fluxo de trabalho é composta de um ou mais trabalhos que podem ser executados sequencialmente ou em paralelo

empregos :

Este fluxo de trabalho contém um único trabalho chamado "build"

construir :

O tipo de executor no qual o trabalho será executado

executa em : ubuntu-latest

As etapas representam uma sequência de tarefas que serão executadas como parte do trabalho

passos :

Faz check-out do seu repositório em \$GITHUB_WORKSPACE, para que seu trabalho possa acessá-lo

- usos : ações/checkout@v3

Executa um único comando usando o shell do runner

- nome : Executa um script de uma linha

executar : eco Olá, mundo!

Executa um conjunto de comandos usando o shell do runner

- nome : Executa um script multi-linha
correr : |
echo Adicionar outras ações para construir,
teste de eco e implante seu projeto.

Finalizamos mais uma unidade 😊

Espero que tenham absorvidos todos
esses aprendizados e colocado a mão
na massa testando as ferramentas
indicadas.

#FocoNosMétodosÁgeis

Testando os aprendizados

Que tal testar os aprendizados desta unidade? Vem comigo!

Com base nos estudos realizados nesta unidade, marque VERDADEIRO ou FALSO nas perguntas/afirmativas abaixo:

O Lean se concentra em melhorar a eficiência através da eliminação de desperdícios no processo de desenvolvimento de software.

☐ Verdadeiro

☐ Falso

SUBMIT

A programação em pares no XP consiste em dois desenvolvedores escrevendo código simultaneamente em diferentes partes do projeto.

☐ Verdadeiro



Falso

SUBMIT

Sintonia do Saber

Mudar nunca foi algo fácil, muito pelo contrário, qualquer mudança independente do impacto ou risco pode se tornar um grande desafio. Para falar um pouco sobre mudanças, neste episódio, contamos com a presença do Antônio Muniz que iniciou sua trajetória na metodologia tradicional e hoje é adepto dos processos ágeis. Antônio é Head Agilidade e DevOps na SulAmérica.

 [#72 Do Tradicional ao Devops: A Jornada Colaborativa – Os Agilistas](#)

#72 Do Tradicional ao Devops: A Jornada Colaborativa – Os Agilistas

Mudar nunca foi algo fácil, muito pelo contrário, qualquer mudança independente do impacto ou risco pode se tornar um grande desafio. Para falar um pouco sobre mudanças, no episódio dessa semana, contamos com a

presença do Antônio Muniz que iniciou sua trajetória na metodologia tradicional e hoje é adepto dos processos ágeis.

CONSULTE MAIS INFORMAÇÃO OS AGILISTAS >

Leitura Recomendada

Olá, estudantes! 

Um processo de software consiste de um conjunto de atividades realizadas para a sua construção e manutenção. Ao longo dos últimos anos, a crescente demanda por software de qualidade, aliada à pressão em relação aos prazos de entrega fizeram com que a eficiência de muitas técnicas e processos tradicionalmente utilizados passasse a ser questionada.

Diante deste contexto, os processos ágeis como Extreme Programming , passaram a atrair o interesse dos meios acadêmico e industrial. Devido à simplicidade e agilidade adotada na solução de problemas através do uso de eficientes práticas de programação, as equipes que usam XP vêm obtendo sucesso em seus projetos o que vem contribuindo para o aumento da sua popularidade.



Artigo - Uma Investigação sobre o Uso de Práticas Extreme Programming no Desenvolvimento Global de Software.pdf

362.5 KB



Referência bibliográfica do artigo

URDANGARIAN, Roger Gonçalves. **Uma Investigação sobre o Uso de Práticas Extreme Programming no Desenvolvimento Global de Software**. Pontifícia Universidade Católica do Rio Grande do Sul Faculdade de Informática Programa de Pós-Graduação em Ciência da Computação, mês, ano de publicação. Porto Alegre, 2008.

Finalizar Aula 

Download de Conteúdo

Que tal baixar todo esse conteúdo e aproveitar os estudos no seu melhor momento?

Faça o download do conteúdo dessa unidade:



BAIXE O CONTEÚDO DESTA UNIDADE..pdf
5 MB

